

DiscoveryHub

An eDiscovery platform — complete technical walkthrough

Team Questify · 7 services, 10 datastores, 9 Kafka topics

Verified against the code and a running stack.

Where a document in the repository disagrees with the code, the code won and the disagreement is called out.

Contents

1. The problem: what eDiscovery actually is
2. The lifecycle the system models
3. Architecture at a glance
4. The system by the numbers
5. The shared contract: the message schema
6. Two identifiers, and why dedupe is subtle
7. P1 — Ingestion :8081
8. P2 — Storage / Archive :8082
9. P3 — Search :8083
10. P4 — Case :8084 and Hold :8086
11. Legal hold: five guards between a candidate and deletion
12. P2.2 — Retention and Disposition :8087
13. P2.2 — API and watching a sweep
14. P5 — Evidence Export and Chain of Custody :8085
15. Kafka: the nine topics and what flows over them
16. Datastores and who owns them
17. The frontend: designed around services being down
18. The corpus, and the traps it plants on purpose
19. Running it — two modes, and when to use each
20. Sharing one stack — and the delete button you are sharing with it
21. Traps that have already cost real time
22. Testing and verification
23. Current state of your running stack
24. A demo script that exercises the whole system
25. Design decisions worth defending

01 The problem: what eDiscovery actually is

When an organisation is sued or investigated, it must produce its own internal communications as evidence. DiscoveryHub is the system that makes that possible, and — just as importantly — provable.

The hard part is not storing email. It is that four obligations pull against each other, and the system has to satisfy all of them at once:

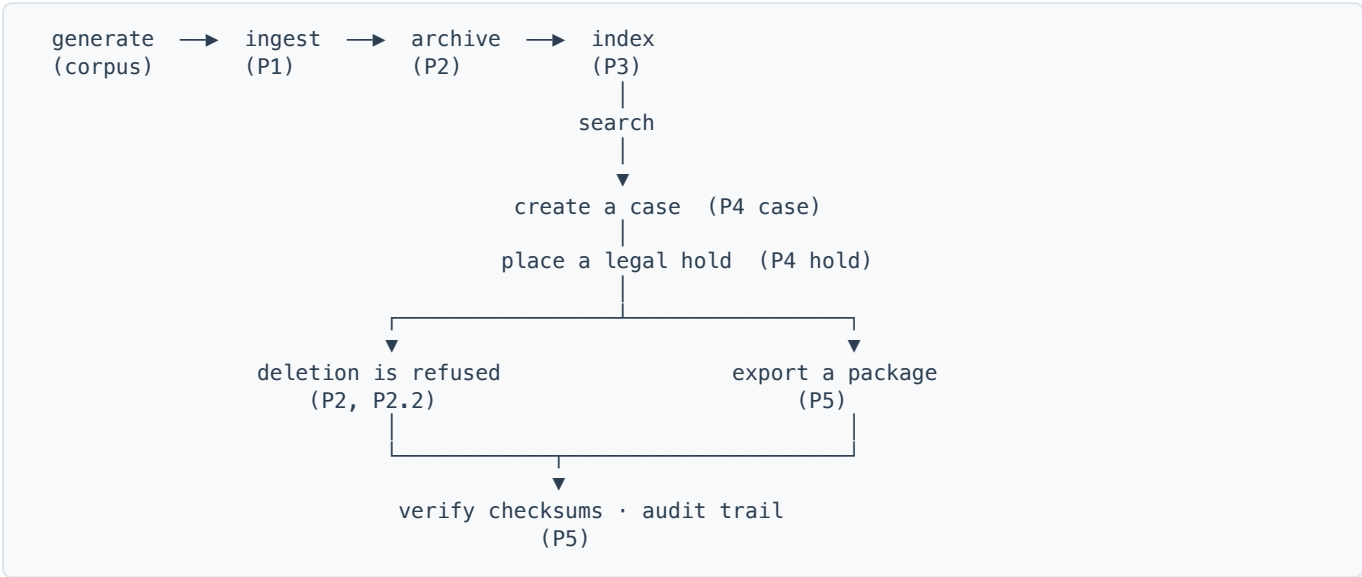
Obligation	What it demands	What it fights with
Preservation	Once litigation is foreseeable, relevant messages must not be destroyed	Retention — which exists to destroy data on a schedule
Retention	Data past its legal retention period must be disposed of	Preservation — a hold freezes the clock
Production	Hand the other side a complete, unaltered evidence set	Volume — 12,000 messages is a small matter
Defensibility	Prove what you did, when, and who did it	Everything — every action has to leave a record

The consequence of getting this wrong is not a bug report. Destroying evidence under a legal hold is spoliation: sanctions, adverse inference instructions, in some jurisdictions criminal exposure. That is why so much of this system's design is about refusing to act when it is not certain.

The single design rule that explains most of the codebase: when the system cannot verify whether a message is protected, it treats the message as protected. Every hold path fails closed. Over-preserving costs storage; under-preserving costs the case.

02 The lifecycle the system models

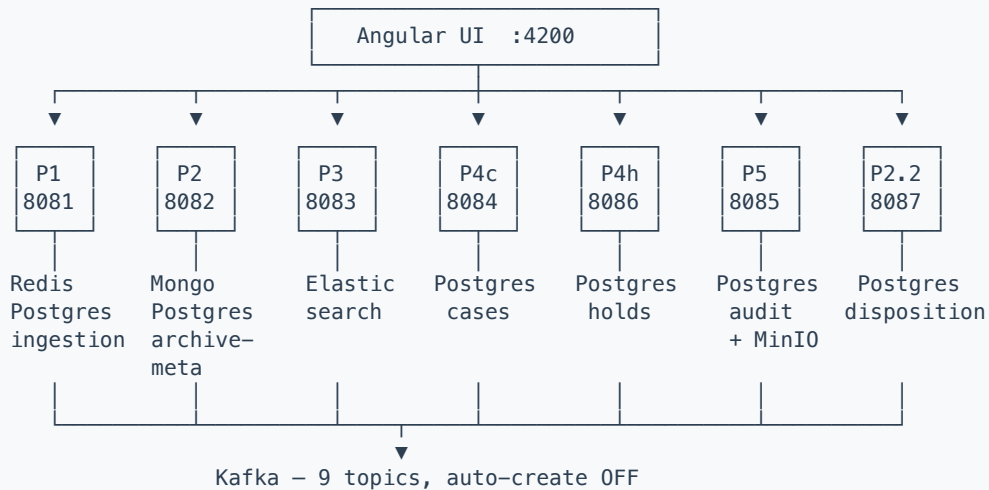
Every service exists to serve one step of a single end-to-end flow. This is the demo script, and it is also the architecture:



Step	Service	Port	Requirement
Accept communications, dedupe, publish	P1 ingestion-service	8081	FR-1
Store durably — system of record	P2 storage-service	8082	FR-1, FR-4.6
Index and full-text search	P3 search-service	8083	FR-3
Case lifecycle, custodians, evidence	P4 case-service	8084	FR-2
Legal hold, scope resolution	P4 hold-service	8086	FR-4
Evidence export + chain of custody	P5 export-service	8085	FR-6, FR-7
Retention and disposition	P2.2 disposition-service	8087	FR-5
Operator UI	frontend (Angular)	4200	FR-8

03 Architecture at a glance

Seven Spring Boot services, each owning its own datastore, coordinating over Kafka for anything asynchronous and HTTP for anything that must be answered now.



- **Stateless at the edge.** P1 accepts and publishes but never stores. FR-1.4 requires that the component accepting messages is not the component storing them, so an archive outage queues in Kafka instead of losing data.
- **One datastore, one owner (NFR-1).** Six separate PostgreSQL *containers*, not six schemas in one database. No connection string lets one service read another's tables.
- **Async where it can be, sync where it must be.** Ingest, indexing, hold fan-out, disposition and export all flow over Kafka. The hold check before a deletion is a synchronous HTTP call, because a stale answer there destroys evidence.
- **The contracts module is a library, not a service.** It sits outside `services/` deliberately — every service compiles against it and nothing deploys it.

04 The system by the numbers

Java modules	9 + parent	Java source	~17,300 lines
Java tests	515 @Test in 61 files	Frontend source	~4,400 lines TS
Frontend spec files	8	Kafka topics	9
PostgreSQL instances	6	Other datastores	Mongo, Elasticsearch, Redis, MinIO
Flyway migrations	16 across 6 services	REST controllers	13
Corpus messages	12,000 (+25 re-sends)	Custodians	24

Corpus composition, from `fixtures/manifest.json` and reproducible with `--stats` :

Property	Value	Why it is that value
Unique messages	12,000	FR-1.2 floor is 10,000
Deliberate re-sends	25	So a healthy load sends 12,025 and stores 12,000 — FR-1.6 visible
Custodians	24, 337–879 messages each	FR-1.2 floor is 20
Email / chat	7,199 / 4,801 (60/40)	Steered per thread, not sampled — chat threads run longer
With attachments	1,304 (10.87%)	FR-1.3 floor is 5%; real bytes with real SHA-256
Distinct threads	2,880	Type is constant within a thread
Date range	2017-01-03 → 2026-09-04	Spans the retention cutoffs on purpose
Past 7-year retention	2,115	So disposition has something to delete
Privileged messages	5	So the search filter has something real to exclude

The generator is deterministic: output is a pure function of `--seed` , `--count` , `--custodians` and `--duplicates` . `messages.ndjson` must hash to `8e35f5b9...31493a` . That is what lets a demo script reference a specific message and still work on somebody else's laptop.

05 The shared contract: the message schema

One frozen wire format, defined once in `contracts/Message.java`, read by P1, P2, P3 and P5. Renaming or retyping a field is a breaking change needing every owner's agreement.

Field	Type	Req	Notes
messageId	UUID	yes	Ours. Derived from externalId, never random
externalId	string	yes	Source system's id. The idempotency key
source	string	yes	EXCHANGE, TEAMS
type	enum	yes	EMAIL CHAT
custodianId	string	yes	Owner of the mailbox this copy came from — <i>not</i> the sender
from / to / cc	string / string[]	yes	Arrays always present, possibly []
subject	string	no	Absent on CHAT
body	string	yes	Plain text; replies may quote history
sentAt	ISO-8601 UTC	yes	Drives every retention decision
threadId / inReplyTo	UUID	yes / no	Conversation grouping; inReplyTo absent on openers
attachments	Attachment[]	yes	May be empty
labels	string[]	yes	PRIVILEGED, SENSITIVE

- **Absent and empty are different on the wire.** Optional fields are omitted entirely rather than sent as `null`; array fields are always present.
- **contentBase64 is ingestion-path only.** Once P2 has the bytes it is dropped, and from then on `sha256` is what everything downstream reasons about. It must never appear in search results, in P2's read API, or in an audit event.
- **Attachment filenames are untrusted.** Not unique, not sanitised — treat as hostile when building export paths.

06 Two identifiers, and why dedupe is subtle

`externalId` is the source system's key and the only thing dedupe operates on. `messageId` is ours, and it is what cases, holds, search hits, export manifests and audit records all reference. It is derived, never generated:

```
messageId = UUID.nameUUIDFromBytes("message:" + externalId) // Ids.messageId()
```

Two independent keys can each make a message a duplicate. Either way it is dropped, the drop is an audit event, and the API returns 2xx — a source system re-sending is normal operation, not a client fault.

Key	Catches	Enforced where
externalId	The same record submitted twice	Redis SETNX in P1 (fast path); UNIQUE constraint in P2 (guarantee)
contentHash	The same message under a <i>different</i> source key — a re-export, a re-crawl, two connectors on one mailbox	ContentHash.of() in contracts, called by both P1 and P2

The fingerprint covers custodianId, source, type, from, to, cc, subject, body, sentAt, threadId, inReplyTo and each attachment's filename, size and sha256. Fields are length-prefixed so content cannot forge a field boundary. It excludes the two identifiers, and it excludes `labels` because classification is mutable.

`custodianId` is inside the fingerprint, and that is the whole design. The same conversation captured from two mailboxes is *not* a duplicate: two externalIds, two messageIds, identical body. Both must be kept, because a hold on one custodian has to preserve their copy independently of the other's. Hashing content alone would collapse them and silently destroy evidence. The corpus plants this case on the narrative's smoking-gun message specifically so a naive dedupe breaks loudly.

Scenario	Outcome
Same content, same mailbox, different externalId	Duplicate — dropped
Same content, different mailbox	Two records — both kept

07 P1 — Ingestion :8081

Accepts communications, validates them, dedupes them, publishes them. It does not store them.

Endpoint	Purpose
POST /messages	JSON array batch. Not atomic — per-item outcomes
POST /messages/upload	multipart; JSON array or NDJSON
POST /messages/upload?async=true	202 with a job id, for large files
GET /messages/uploads/{jobId}	Poll an async upload
GET /messages/{externalId}/status	Has this id been ingested?
GET /messages/stats	Counts since this instance started

Outcome	Meaning	HTTP
ACCEPTED	Published to messages.ingested	200
DUPLICATE	externalId already seen. Dropped, audited, not an error	200
REJECTED	Failed validation, with a reason	200
FAILED	Infrastructure failure; retry the message	503

- **A batch is not atomic.** A duplicate in the middle of a batch must not reject the other 199. Even an element Jackson cannot decode becomes one REJECTED with the offending field named, and its externalId is recovered from the raw JSON so a loader knows what to fix.
- **Dedupe is claim-then-publish.** If publishing fails after a claim, the claim is *released* — otherwise a transient broker error would mark the id as seen and silently swallow the retry.
- **Redis fails open.** P1 still starts without Redis, health still reports UP, and duplicates pass through to be caught by P2's unique constraint. Slower dedupe beats refused traffic.
- **Attachments are verified at the boundary.** sha256 is recomputed from contentBase64 and sizeBytes checked against the decoded length. Trusting the client's hash would surface much later as a failed export verification.
- **Size caps are refused whole, with 413.** Over `max-batch-size` (1000) or `max-request-bytes` (16 MB). A client given a partial result cannot tell which messages were dropped. The body cap counts bytes when there is no Content-Length, because a chunked request declares none and a cap that only reads the header is no cap at all.

Point probes at `/actuator/health/readiness`, not `/actuator/health`. Only Kafka can make P1 unready — publishing blocks on the broker ack before the API says ACCEPTED. Redis deliberately cannot: the dedupe store fails open, so its indicator stays UP and reports `dedupe: degraded`. Boot's stock Redis contributor is disabled because it does the opposite, and a probe reading it would evict the very instance that failing open exists to keep serving.

Measured against the full corpus

Accepted / deduped / rejected / failed

12,000 / 25 / 0 / 0

Throughput

~151 msg/s (80 s for the full load)

Second identical load

0 new messages published; 12,025 deduped in 1.9 s

That 80 s → 1.9 s gap locates the bottleneck precisely: `publishIngested` blocks on the broker ack for every message, and the dedupe short-circuit skips it entirely. Batching the sends is the obvious optimisation, and it does not matter yet.

08 P2 — Storage / Archive :8082

The system of record. Consumes ingested messages, stores them durably, and enforces the legal-hold guard on every path that deletes anything.

What	Where	Why there
Message content — headers, body, labels, thread, attachments <i>including bytes</i>	MongoDB, one document per message	Write-once, never updated
Hold state + retention bookkeeping — <code>on_hold</code> , <code>hold_count</code> , <code>retention_override_at</code>	PostgreSQL, one slim row per message	Changes on every hold; queried by every sweep

The split is deliberate. Keeping hold state out of the Mongo document means a hold update never has to touch — or race with — the message content, and the slim Postgres row keeps the disposition candidate query cheap regardless of message size.

- **Mongo document first, Postgres row second, on write.** `save` keyed by `messageId` is an idempotent upsert, so losing a race on `UNIQUE(external_id)` afterwards leaves nothing incorrect behind. The other order risks a hold-status row claiming a message no document backs.
- **Postgres row first, Mongo document second, on delete.** The row goes inside the transaction and the document after; if the Mongo delete throws, the transaction rolls back and the row returns. The two stores are never left disagreeing about whether a message exists.
- **`findByIdForUpdate`, not `findById`.** Without the row lock, a concurrent hold event could land between the P4 check returning 'not held' and the delete that follows.
- **No update endpoint, by design.** An archived message is write-once.

Resiliency — an outage must not lose messages (NFR-2):

- `ack-mode: record` — an offset is committed only after its listener returns normally.
- A `DefaultErrorHandler` with `unbounded` fixed backoff (retry every 3 s, forever). A transient failure parks the consumer on the failing record rather than skipping ahead, so retained records are processed in order once the service is healthy.
- Expected 'bad' records never reach the error handler: malformed JSON is logged and skipped, and a `DataIntegrityViolationException` from the unique constraint is recorded as a dedupe — the idempotency guarantee working, not a failure.

The Mongo config prefix is `spring.mongodb`, not `spring.data.mongodb`. Boot 4.1 renamed it. The old prefix still parses as valid YAML, shows up in `/actuator/env`, starts clean and connects — and every write silently lands in the driver's fallback `test` database instead of the configured one.

Never delete from one store by hand. `DELETE FROM message_hold_status` in psql leaves the Mongo document, so the archive still serves content the sweep no longer knows about. Every deletion must go through `MessageDeletionService` or `DispositionCommandListener` — the only two places that keep the stores in step and the only two that enforce the hold guard.

09 P3 — Search :8083

Consumes `messages.archived`, indexes into Elasticsearch, and serves the query surface the investigator actually works in.

Endpoint	Purpose
GET /search · POST /search	Query. Criteria: q, custodianIds, type, from, labels, sentAfter, sentBefore, hasAttachment, onHold
GET /search/history · DELETE /search/history	Executed searches, newest first
POST /search/saved · GET /search/saved · GET,DELETE /search/saved/{id}	Saved searches
POST /search/saved/{id}/run	Re-run a saved search
POST /search/add-to-case · POST /search/add-selected-to-case	File results as evidence (via Kafka to P4)

- **A criterion-less query is a 400, not everything.** The index holds the whole corpus; a screen that opens with 12,000 messages has answered a question nobody asked and buried the one you came to ask.
- **History is a side effect of searching, not a POST.** P3 writes to its own `search-history` index when a search executes — first pages only, since paging is the same question again. There is no write endpoint, so a client cannot invent history it never executed.
- **Filing to a case is eventually consistent.** `add-to-case` publishes an event that case-service consumes; the response counts what was collected, not what has been written. The UI says 'queued' rather than implying otherwise.

Elasticsearch is pinned to 9.4.x to match the client Spring Boot 4.1.1 ships. The client enforces a server version check, so an 8.x server rejects it outright.

10 P4 — Case :8084 and Hold :8086

P4 is two deployables, not one. Case and hold are separate bounded contexts with separate databases, coordinating over `cases.events` and one synchronous endpoint.

case-service :8084	hold-service :8086
POST/GET /cases, GET/PATCH /cases/{id}	POST /holds, GET /holds/{id}
POST /cases/{id}/transitions	POST /holds/{id}/release
GET/POST /cases/{id}/custodians	GET /holds/check?messageId= — the guard P2 calls
GET/POST /cases/{id}/evidence, POST .../evidence/batch	GET /holds/covering?messageId=
DELETE /cases/{id}/evidence/{messageId}	GET /holds/active — holds as scope
POST /cases/evidence/lookup, GET /cases/stats	POST /holds/evidence-check, GET /holds/stats

The split is why the UI shows two badges rather than one 'P4'. A case list that loads while every hold check fails is a real state, and one badge could not express it. Closing a case publishes to `cases.events`; hold-service consumes it and releases that case's holds.

Overlapping holds — the part that is easy to get wrong.

Two holds routinely cover the same message: 'Rahul's mailbox' and 'the Phoenix investigation' are different scopes on different cases that overlap on whatever Rahul sent about Phoenix. Releasing one does not unprotect the message; only the last release does. P4 owns that decision in one place, `HoldReleasePlan`: a release publishes `held=false` only for messages no other ACTIVE hold covers, and the `hold.released` audit records both `unprotectedMessages` and `stillHeldMessages`.

- `held=false` is state, not a decrement. P2's mirror sets `on_hold = false, hold_count = 0` outright, because P4 has already established nothing else covers the message. A decrementing mirror would never reach zero for a message that was ever held twice — it would sit at 1, permanently on hold, permanently exempt from retention.
- `GET /holds/check` was always right about overlap. It joins coverage to ACTIVE holds, so it refused deletes even while the published events were wrong. That is why the bug was survivable — and why it was invisible: nothing was ever deleted, the flags were just quietly wrong.
- `GET /holds/covering` answers the question that follows. An investigator who releases their hold and finds the message still held wants to know *which* hold is responsible.

A new hold protects nothing yet. `POST /holds` returns 202 and a `RESOLVING` hold; a worker expands the scope over `holds.commands` and flips it to ACTIVE with a `messageCount`. FR-4.3 requires this propagation to be asynchronous — which is precisely what creates the window that disposition guard 2 exists to close.

11 Legal hold: five guards between a candidate and deletion

They are redundant on purpose. Each catches something the others structurally cannot.

#	Guard	Where	Catches
1	<code>on_hold</code> flag	P2's row, mirrored from <code>holds.events</code>	Almost everything, for free
2	Active hold scope	GET <code>/holds/active</code> — once per run	A hold on a case P4 has not yet expanded to messages
3	Held-case evidence	POST <code>/holds/evidence-check</code> — once per run	A message attached to a held case that the hold's own scope does not cover
4	<code>GET /holds/check</code>	Synchronous, per message	The backstop for anything neither run-level answer expressed
5	<code>AND on_hold = false</code>	Inside the DELETE statement	A hold placed <i>after</i> the check, milliseconds before the write

Only the fifth is evaluated atomically with the write, which is why it exists even though four checks already passed. A message refused there is recorded as `SKIPPED_HOLD` and audited as `disposition.refused` — the demonstrable proof FR-4.6 asks for.

Why guard 2 exists. Guards 1, 3 and 4 all read per-message state, and FR-4.3 makes propagation asynchronous. Between an investigator placing a hold and every message in scope being flagged, every per-message signal correctly reports 'not held'. A sweep landing in that window would destroy exactly the evidence the hold was placed to preserve. Guard 2 evaluates the hold's *scope* — custodians, dates, case — so the hold is enough, expanded or not.

- **Custodians match exactly, and an *empty* custodian set means every custodian** — a hold placed without narrowing to people covers the whole corpus. If you mean 'no one', do not return the hold.
- **Date ranges are exact and inclusive**, either end may be unbounded.
- **Search terms are not evaluated.** This service has identities and timestamps, not bodies. A term-scoped hold therefore protects everything within its custodian and date range — over-protecting by design. Under-protecting means deleting evidence and being unable to say so; over-protecting means a message survives one cycle longer. Not comparable costs.

Why guard 3 exists. A hold's scope and a case's contents are different sets. A hold is written as custodians plus dates, but FR-2.4 lets an investigator add *any* message to a case. A hold covering two custodians for 2019–2021 genuinely does not cover a 2024 message from a third custodian — and yet if a human pulled that message into the matter, deleting it destroys part of a production someone already selected. Scope alone would delete out-of-scope evidence; evidence alone would delete everything a broad custodian hold was placed to freeze before anyone reviewed it.

Evidence membership protects only while the case is under hold. A message in a case with no hold stays deletable — otherwise adding anything to any case would silently switch retention off for it, and retention would decay to nothing as the system got used.

Fail closed, and partial knowledge counts as no knowledge. `HoldContext` keeps 'nothing is under hold' and 'P4 could not be asked' as distinct states in the type, because an empty collection is the most dangerous available reading of a network error. If the scopes come back but the evidence check fails, the whole context is unavailable — knowing scopes but not evidence membership would let the sweep delete an out-of-scope evidence item with full confidence.

12 P2.2 — Retention and Disposition :8087

Owns the retention policy, the sweep that destroys expired messages, and the ledger that proves what it did.

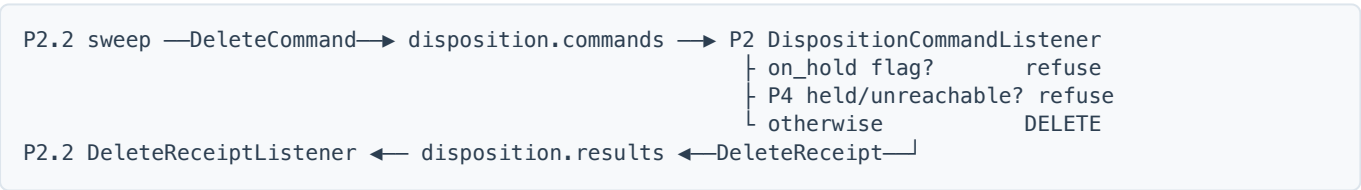
- Reads the retention period per communication type from **its own database** (FR-5.1).
- Asks P2's archive for messages whose `sentAt` is past their type's cutoff.
- Checks every hold guard; skips anything covered.
- Deletes the rest and records every decision — deleted, skipped, failed — in a ledger that outlives the messages themselves.

Eligibility is computed per run, never stored per message. Change a retention period and the next sweep picks it up with no backfill. The policy lives in the `retention_policies` table rather than `application.yml`, because a restart to change a number is not configurable in any useful sense, and a value that exists only in a container's environment cannot be shown in the UI or audited when it changes.

```
curl localhost:8087/retention/policies

# ISO-8601 durations. P2555D = seven years, PT2M = two minutes.
curl -X PUT "localhost:8087/retention/policies/EMAIL?actor=saketh" \
  -H 'Content-Type: application/json' -d '{"period":"PT2M}"'
```

How it deletes — the command/receipt loop. P2 owns the messages, and NFR-1 says one datastore one owner, so P2.2 does not write to them:



The sweep records `DELETE_REQUESTED`, which is all it can honestly claim at that moment — the message is not gone, it has been asked about. The receipt settles the row to what P2 actually did.

Receipt	Ledger becomes	Reading
DELETED	DELETED	Gone, confirmed
REFUSED_HOLD	SKIPPED_HOLD	A hold landed between the check and the delete, and P2's own guard caught it. The rarest and most reassuring outcome in the system
NOT_FOUND	DELETED	Already absent. The desired state holds; recording a failure would make the next run look like it is retrying something broken
FAILED	FAILED	Still there. The next sweep finds it expired again and retries

- **Settling is guarded.** Only a row still awaiting a receipt can move, so a redelivered receipt on an at-least-once topic cannot turn a recorded refusal back into a deletion.
- **A row still at `DELETE_REQUESTED` hours later means P2 is not consuming.**
- **The sweep is deliberately not `@Transactional`.** A rollback would erase the ledger record of deletions that already happened in P2's database, which no transaction of ours can undo. The ledger has to survive the

failure that makes it interesting.

- **A second concurrent run is refused with 409, not queued.** Two sweeps would evaluate the same candidates and race on the same rows, and whatever a dropped tick skipped is still expired at the next one.

13 P2.2 — API and watching a sweep

Endpoint	Purpose
POST /disposition/runs	Sweep now, synchronously. <code>?dryRun=true</code> decides everything, deletes nothing
POST /disposition/runs?async=true	202 with a QUEUED run id
GET /disposition/runs/stream	Live progress as server-sent events; closes when the run finishes
GET /disposition/runs/progress	The same snapshot, polled once
GET /disposition/runs · /runs/{id} · /runs/{id}/items	History, summary, per-message ledger. <code>?outcome=SKIPPED_HOLD</code> is the 'what did holds save?' view
GET /disposition/messages/{messageId}	Every decision ever recorded about one message. Survives the message
GET /disposition/cases/{caseId}/protected	Everything this case's holds have saved, across every run
GET /retention/policies · PUT /retention/policies/{type}	Retention policy (FR-5.1)
GET /disposition/stats · /stats/candidates	Dashboard counts; what the next sweep would touch

A sweep makes one HTTP call to P4 per candidate and considers up to `batch-size` of them, so the synchronous form holds the connection for the length of the work — over the full corpus that is exactly the bulk operation NFR-3 says must not time out the UI. Start it asynchronously and watch it:

```
curl -X POST "localhost:8087/disposition/runs?async=true&actor=sahithi"
curl -N localhost:8087/disposition/runs/stream

event:progress
data:{"runId":"37956dfd...", "status":"RUNNING", "processed":3, "total":5,
      "deleted":1, "skippedHold":2, "percent":60, "terminal":false}
```

- The current snapshot is sent on connect, so a client attaching mid-sweep — or just after one finished — sees state immediately rather than an empty stream.
- Progress is in memory and for the current run only. The ledger is the durable record; a per-message counter written to the chain-of-custody table would be write traffic for something nobody reads tomorrow.
- `blocking_hold_id` and `blocking_case_id` are **copied onto the ledger row**, not joined from P4, so the protection record outlives both the release of the hold and the closing of the case.

`/cases/{caseId}/protected` is the money endpoint for a demo. It answers 'what did this hold actually save?' with per-message evidence, across every run, after the fact.

14 P5 — Evidence Export and Chain of Custody :8085

Two responsibilities, one owner, one datastore: prove what happened (FR-7), and hand over a package that can be proven unaltered (FR-6).

Chain of custody. Every service publishes to `audit.events`; P5 is the only consumer and the only durable store. It is insert-only — there is no update or delete endpoint anywhere in the service, and that absence is the entire enforcement mechanism for FR-7.3.

Export, and why it can never produce a partial package:

- `POST /exports` does the minimum synchronous work: validate, persist a QUEUED row, publish the job id, return 202 with a Location header. The build happens on the `export.jobs` consumer thread.
- `process` is idempotent against redelivery — a job already past QUEUED is left alone, so a rebalance cannot rebuild or clobber an existing package.
- The packages bucket gains an object only once the whole package is built successfully *and* copied server-side from `export-staging` to `export-packages`. If anything fails before that copy, the job is FAILED, staged objects are discarded, and nothing ever appears under the job's key.
- `POST /exports/{jobId}/retry` is accepted only for a FAILED job and starts the build over. It cannot duplicate or corrupt, because there is nothing to conflict with until the retry itself succeeds.

Checksums — the part that makes it evidence rather than a zip file:

- Every attachment's bytes are **re-fetched from P2 and re-hashed**, never trusted from the archive's own metadata. If the fetched bytes do not match the recorded sha256, the export refuses outright — that is the chain of custody breaking.
- `manifest.json` inside the package lists every message and attachment with the checksum computed from the exact bytes written into the package.
- `GET /exports/{jobId}/verify` re-downloads the package and re-derives every checksum **from its own bytes, with no call back to P2**. That independence is what makes tampering detectable rather than merely claimed to be detectable.

```
JOB=$(curl -s -X POST localhost:8085/exports -H 'Content-Type: application/json' \
  -d '{"caseId":"demo","custodianId":"cust-001"}' | jq -r .jobId)
curl -s localhost:8085/exports/$JOB          # QUEUED → RUNNING → COMPLETED
curl -s localhost:8085/exports/$JOB/verify  # re-hash from the delivered bytes
curl -s "localhost:8085/audit?action=export.completed"
```

15 Kafka: the nine topics and what flows over them

Topic	From → To	Carries	Parts
messages.ingested	P1 → P2	Accepted, deduped, not yet stored	6
messages.archived	P2 → P3	Written to the archive, ready to index	6
holds.commands	P4 → its own workers	Fan-out over a hold scope too large to resolve synchronously	3
holds.events	P4 → P2, P3	A custodian or message came under hold, or came off it	3
disposition.commands	P2.2 → P2	One message past retention, to be removed. A request, not a warrant	3
disposition.results	P2 → P2.2	What actually became of a DeleteCommand	3
cases.events	case-service → hold-service	Case created, transitioned, closed → release its holds	3
export.jobs	P5 → its own workers	One export job to build	3
audit.events	everyone → P5	Chain of custody	6

- **Auto-create is off.** A typo fails loudly instead of quietly creating a one-partition topic nothing produces to. New topics go in `infra/kafka/create-topics.sh` and in `Topics.java`.
- **Kafka advertises two listeners:** `kafka:19092` for containers, `localhost:9092` for services run from your IDE. Use the right one for where the process actually lives.
- **A Homebrew Kafka on 9092 is a silent disaster.** Two brokers means your producer succeeds, your consumer receives nothing, and both configs look correct. `brew services stop kafka`.

disposition.commands is a loaded gun if you are not careful. During testing, a ten-second cron completed two sweeps of `candidates=500, deleted=499` before anyone could intervene. It was in KAFKA mode, so the commands were *published* rather than executed and the corpus survived — then sat on the topic waiting for whoever added P2's consumer with `auto-offset-reset: earliest`. The topic had to be deleted and recreated. That consumer now exists, so the reprieve is gone: a stray command is a real deletion, subject only to P2's hold guard.

16 Datastores and who owns them

Port	Store	Owner	Holds
5437	PostgreSQL <code>ingestion</code>	P1	message_id_map — the idempotency backstop
6379	Redis	P1	Dedupe keys, 7-day TTL. Fails open
27017	MongoDB <code>archive</code>	P2	Message content, attachment bytes included
5433	PostgreSQL <code>archive</code>	P2	message_hold_status — hold state + retention bookkeeping only
9200	Elasticsearch	P3	communications, search-history, saved-searches
5434	PostgreSQL <code>cases</code>	P4 case	Cases, custodians, evidence items
5436	PostgreSQL <code>holds</code>	P4 hold	Holds and hold_coverage
5435	PostgreSQL <code>audit</code>	P5	Audit log (insert-only) and export jobs
9000	MinIO	P5	export-staging, export-packages, archive-attachments
5438	PostgreSQL <code>disposition</code>	P2.2	Retention policies, runs, the per-message ledger

- **Schema comes from Flyway migrations, never from Hibernate.** `ddl-auto` stays at `validate`, so booting the context is itself an assertion that migrations and entities agree.
- **Inside the compose network every Postgres is on 5432.** Only the *host* ports differ, so no service's connection string ever moved when they were renumbered.
- `down -v` deletes your data. Plain `down` keeps the volumes.

The two host ports that catch people out: 5437 and 5438. 5437 belonged to disposition before P1 gained a database and took it; disposition moved to 5438. A branch that predates the move will quietly point P2.2 at P1's Postgres — and it will start, because the failure is semantic rather than a connection error.

17 The frontend: designed around services being down

Angular on :4200, five pages, talking to seven ports. It works with any subset of the services running, including none.

Page	Route	Reads
Dashboard	/home	All of them, for counts and recent activity
Search	/search	P3 :8083
Cases & Legal Hold	/cases	P4 case :8084, P4 hold :8086, P2.2 :8087
Retention & Disposition	/retention	P2.2 :8087, P2 :8082
Exports & Audit	/export-audit	P5 :8085, P4 case :8084

- **One request per panel, and the panel owns its failure.** Every region is an `<app-panel>` wrapping one `httpResource`, rendering its own loading, error and retry. Six panels against three services means five working and one explaining itself. No global error handler, no toast.
- **No route resolvers.** A resolver fetches before activating a route, so one slow service would hold up a page whose other panels are fine.
- **Everything has a deadline.** 8 seconds by default, 3 for search, 30 for package verification because it re-hashes a zip. A service that is *down* fails in milliseconds; one that is *wedged* holds a spinner forever.
- **Live progress degrades to polling.** A sweep's SSE stream that fails before delivering anything falls back to `/disposition/runs/progress`, so a buffering proxy costs latency rather than the progress bar.

'Empty' and 'could not ask' are never the same thing. This is a correctness property, not a UX one. Upstream, an unreachable hold-service means *held* — both P2 and P2.2 fail closed. So a hold panel rendering a failed request as 'no holds are in force' would state the exact inverse of what the system is acting on. `failure.ts` classifies a refused connection as *unreachable* and names the service and port.

`Resource.value()` throws in the error state, and `defaultValue` does not cover it — it only covers idle and loading. Angular instantiates projected content eagerly, so a panel's table is evaluated while the panel is showing a failure, and the throw blanks the whole page. Every list goes through `valueOr()`, and `pages.spec.ts` mounts every page with every service down to keep it that way. This has been introduced three times.

CORS is two settings, not one. Every service allows any loopback port in its own `CorsConfig` — do not pin 4200, because an IDE preview pane and `ng serve --port 4201` are different origins. Actuator endpoints use a *separate handler mapping* that ignores `addCorsMappings`, so `management.endpoints.web.cors` is set in every `application.yml` too. Miss that one and the status strip shows every service DOWN while every panel loads fine.

18 The corpus, and the traps it plants on purpose

There is no real communications source, so the corpus is generated — deterministically, and seeded with cases that make specific bugs visible instead of silent.

#	Planted case	The bug it exposes
1	25 deliberate re-sends at the end of the file	A load that stores 12,025 has broken dedupe. Posting the file twice must still leave 12,000
2	The same conversation captured from two mailboxes	Any dedupe keyed on body hash, or on from+subject+sentAt, collapses them and destroys evidence. Planted on the smoking-gun message so the demo breaks loudly
3	A PRIVILEGED-labelled thread	Gives the search filter something real to exclude
4	2,115 messages past the 7-year default, under custodians who later go on hold	Gives disposition something to delete and the hold guard something to refuse

Project Halyard — the demo narrative, planted across 7 threads in 2024. Meridian Dynamics bids for the Northgate transit contract, obtains the incumbent's indicative figures through a recent hire, prices just underneath, wins, and is then asked to 'do some housekeeping' on the evidence. Legal is looped in late, and that exchange is privileged.

- Principals: Dana Whitfield (cust-001), Marcus Ellery (cust-002), Priya Raghunathan (cust-003), Owen Castellanos (cust-004), Renee Toussaint (cust-005).
- Searching **Halyard** surfaces the spine of it — 8 mentions.
- The centrepiece for export and checksum verification is **bridgeline-indicative-figures.csv**.
- The smoking gun — subject **Housekeeping**, 11 May 2024 21:37 UTC — was sent at the weekend and **exists in two mailboxes**.

That narrative is why the demo has something specific to point at in every step: generate → ingest → search → create case → place hold → prove deletion is blocked → export → verify checksums → show the audit trail.

19 Running it — two modes, and when to use each

Everything in Docker. One command, nothing installed but Docker. Best for a demo, a clean-machine check, or sharing a stack.

```
docker compose --profile app up -d --build
# builds 8 images, starts 20 containers, UI on http://localhost:4200
```

Datastores in Docker, services from jars. Best when you are editing code: a change costs a restart instead of an image rebuild, and you can attach a debugger.

```
docker compose up -d          # datastores only
./infra/smoke-test.sh         # reachable *from the host* — where your jar runs
mvn -q package -DskipTests

for s in ingestion storage search case hold export disposition; do
  java -jar services/$s-service/target/$s-service-0.1.0-SNAPSHOT.jar &
done

cd frontend && npm start      # ng serve, live reload
```

Load the corpus once P1 is up — 12,000 messages, about 90 seconds through Kafka into P2 and P3:

```
curl -F "file=@tools/corpus-generator/fixtures/messages.ndjson" \
  "http://localhost:8081/messages/upload?async=true"
```

smoke-test.sh checks the host ports specifically, and that is the point. A container reporting healthy only proves it can talk to itself. A container can be healthy with its port *unpublished* — if a bind loses a race, **docker port** prints nothing while every jar dies on 'connection refused'. **docker restart** does not repair it; **docker compose up -d --force-recreate <svc>** does, and the named volume means no data is lost.

20 Sharing one stack — and the delete button you are sharing with it

Nothing syncs between machines. `docker compose up` on a second laptop creates a second, empty set of databases in its own volumes — Docker packages services, it does not connect hosts. A case is shared only because everybody is pointed at the one machine that owns it.

```
docker compose up -d <datastores>
./infra/serve-lan.sh           # prints the URL to hand round
```

- `frontend/public/api-config.js` derives the service host from `window.location.hostname`, so a visitor's browser calls the host's services instead of their own empty machine.
- `DISCOVERYHUB_WEB_CORS_ALLOWED_ORIGINS` adds the host's LAN address to the allowed origins — one variable covers both the panels and the actuator status strip.

Sharing the link shares the delete button. The retention page runs a real sweep. This has already happened once: a MANUAL, non-dry-run sweep took 486 messages out of the archive, and holds saved only the 14 they covered. The deletion is permanent, and P1 refuses the re-upload as duplicate because dedupe outlives the message. Restoring means clearing Redis *and* deleting the orphaned `message_id_map` rows in postgres-ingestion, then re-uploading.

There is no configuration that makes this safe. `schedule.enabled` is already false and that only stops the *timer* — `POST /disposition/runs` still sweeps on demand, which is exactly what the button does. `dryRun` is a query parameter on the request, not a setting, so nothing on the host can force it. There is also no authentication anywhere in the stack: whoever can reach the port can delete a case. Say so before handing the URL round, and keep it on a network you trust.

21 Traps that have already cost real time

Symptom	Cause	Fix
You edit a component and nothing changes	The <code>discoveryhub-frontend</code> container serves a stale prebuilt dist on <code>[::]:4200</code> while <code>ng serve</code> binds <code>127.0.0.1:4200</code> ; both start happily	<code>docker stop discoveryhub-frontend</code> before <code>ng serve</code>
Health checks pass but you are debugging the wrong process	A service whose port was taken logged APPLICATION FAILED TO START and exited; something else answers that port	<code>lsof -nP -iTCP:8086 -sTCP:LISTEN</code> , compare the PID's start time
Every service dies on 'connection refused' to a healthy container	The port bind lost a race; the container is healthy with its port unpublished	<code>docker compose up -d --force-recreate <svc></code>
Status strip shows every service DOWN, every panel loads fine	Actuator has its own CORS handler mapping	Set <code>management.endpoints.web.cors</code> too
The whole page blanks	<code>Resource.value()</code> threw in the error state inside eagerly-instantiated projected content	Route every list through <code>valueOr()</code>
Producer succeeds, consumer receives nothing, both configs look right	A Homebrew Kafka is also on 9092	<code>brew services stop kafka</code>
Writes succeed but the data is nowhere	<code>spring.data.mongodb</code> instead of <code>spring.mongodb</code> under Boot 4.1 — lands in the fallback <code>test</code> database	Rename the prefix
Search returns messages the archive does not have	A sweep deletes from P2 without removing the document from P3	Rebuild the index

A sweep that deleted nothing is not evidence that holds work. Every hold path fails closed, so an unreachable or misconfigured P4 produces the *exact same result* as holds doing their job: every candidate skipped, refusals throughout the audit trail, no errors anywhere. Before believing a quiet sweep, check `holdScopeAvailable` in the candidates preview and P2's logs for `hold check failed`. The first end-to-end run of the delete loop refused all five test messages for exactly this reason — `CASES_BASE_URL` was unset in P2's container.

- **We do not inherit `spring-boot-starter-parent`**, so its defaults are missing and each absence fails late, looking like something else. Three so far: the `repackage` goal unbound ('no main manifest attribute'), `spring-boot-flyway` absent so migrations silently never ran, and `-parameters` off so every `@PathVariable` threw a 500 at request time. Expect more of the same shape.
- **Spring Boot 4 splits auto-configuration into per-technology modules.** A library on the classpath is not enough — check it is actually *doing* something, not merely present. `DataSourceProperties` also moved package, and the failure is a bare 'cannot find symbol' that looks like a missing dependency.
- **Declaring a `DataSource` bean disables Boot's datasource auto-configuration entirely**, so both of P2.2's datasources must be declared by hand. Omitting `@Primary` surfaces as an `EntityManagerFactory` failure

mentioning nothing about datasources.

22 Testing and verification

```
mvn -q package -DskipTests      # build all modules
mvn package                      # build and run every test
cd frontend && npm run check      # ng build && ng test --no-watch
./infra/smoke-test.sh           # datastores reachable from the host
```

Java tests log ERROR lines on purpose — several assert failure paths ('archive unreachable', 'P2 unreachable', a deliberately injected broker failure). Grep for **BUILD SUCCESS**, not for the absence of errors.

Test	What it actually proves
DispositionIntegrationTest	The sweep against two real Postgres instances via Testcontainers. Booting the context is itself the assertion that Flyway and the entities agree under ddl-auto: validate
DeleteLoopKafkaIntegrationTest	The delete loop against a real broker. The test plays P2: consumes the command, answers with a receipt, asserts the ledger settles — and that a redelivered receipt cannot rewrite it
DispositionCommandIntegrationTest	P2's half: a command on the topic, a row gone from a real archive, a receipt back — and a held message still there
JdbcMessageDeleterTest	AND on_hold = false enforced by a real database (H2 with P2's own DDL). No amount of mocking would show that it works
...BeforeP4HasExpandedIt	The propagation window — guard 2
... EvenWhenItIsOutsideThatHoldsScope	The scope-versus-contents gap — guard 3
pages.spec.ts	Every page mounted with every service down, asserting it still renders and names what is missing
resilience.spec.ts	One service failing leaves the others resolved; a failed panel recovers on reload

One detail worth copying: **DispositionIntegrationTest** builds the archive schema from **P2's own migration file**, read out of **services/storage-service**, rather than from a copy. **JdbcArchiveGateway** hand-writes SQL against another service's table, so the failure it is exposed to is P2 renaming a column — which compiles fine and breaks at run time. A copied fixture would keep passing after such a rename; the real file means the build breaks the moment the contract does.

23 Current state of your running stack

Observed live on 2026-09-14, with all 20 containers up via `--profile app` :

Observation	Value	Reading
Messages in the archive (Mongo)	8,495	Below 12,000 — sweeps have run
Documents in the Elasticsearch index	12,009	Drifted from the archive
Disposition runs recorded	13	totalDeleted 28, totalSkippedByHold 0
Candidates in the next sweep	0	The expired backlog has already been swept
Retention periods in force	EMAIL PT61320H (7y), CHAT PT26280H (3y)	Defaults, unmodified
holdScopeAvailable	true	P4 is reachable — a quiet sweep would be genuine
Delete mode / hold check	KAFKA / required	The safe configuration
Active holds	0	Nothing is currently protected

The 8,495 / 12,009 gap is the known P2↔P3 drift, not something you broke. A sweep deletes from the archive without removing the document from the search index, so Elasticsearch keeps returning hits for messages P2 no longer holds. Expect 'message not found' on some search results until the index is rebuilt.

One documentation discrepancy worth knowing, since it would mislead you: `frontend/README.md` states that `GET /holds/active` and `POST /holds/evidence-check` *do not exist* and that hold-service answers 404 and 405. That is stale. Both were verified answering 200 on your stack, and `holdScopeAvailable` reads `true`. The root README and `DISPOSITION.md` are correct; the frontend README is behind.

24 A demo script that exercises the whole system

Each step proves a different requirement, and the Halyard narrative gives every one of them something specific to point at.

#	Step	What it proves
1	Upload <code>messages.ndjson</code> ; watch accepted 12,000 / duplicates 25	FR-1.6 dedupe, at scale
2	Post the identical file again — zero new messages	Idempotency is real, not claimed
3	Search <code>Halyard</code>	FR-3 index, and the narrative's spine
4	Open the Housekeeping message — two copies, two custodians	Content-hash dedupe did <i>not</i> collapse them
5	Create a case, add those results as evidence	FR-2.4, and the 'queued' honesty of async filing
6	Place a hold on a custodian; watch RESOLVING → ACTIVE	FR-4.3 asynchronous propagation
7	<code>DELETE /messages/{id}</code> on a held message → 409	FR-4.6, demonstrably
8	Drop EMAIL retention to PT2M, run <code>?dryRun=true</code>	FR-5.1 runtime policy, and the blast radius before it happens
9	Run the sweep for real; inspect <code>/runs/{id}/items?outcome=SKIPPED_HOLD</code>	FR-5.3 ledger, and what holds saved
10	<code>/disposition/cases/{caseId}/protected</code>	The protection record, after the fact
11	Export the case; poll to COMPLETED; <code>/verify</code>	FR-6.3/6.5 checksums re-derived from delivered bytes
12	<code>GET /audit</code> filtered by action	FR-7 chain of custody, insert-only

Stop hold-service before step 9 and run it again. Every candidate is skipped and the audit fills with refusals — identical to holds working perfectly. Then show `holdScopeAvailable: false` in the candidates preview, which is the only thing that tells the two apart. That contrast is the single best demonstration of the system's central design decision.

25 Design decisions worth defending

- **Fail closed, everywhere, without exception.** An unreachable hold service means *held*. Five redundant guards stand between a candidate and deletion, and only the last is atomic with the write — which is exactly why it exists.
- **'Empty' and 'could not ask' are different facts**, kept apart in the type system (`HoldContext`), in the UI (`failure.ts`), and in the ledger (`hold_scope_available`). Collapsing them is the most dangerous available reading of a network error.
- **One datastore, one owner.** P2.2 decides what is past retention; P2 decides whether to act and re-checks holds against data it owns. Neither half can be wrong on its own, and with P2 down the commands simply queue.
- **Derived identifiers, not random ones.** The same corpus on five laptops produces five identical sets of ids, which is what makes fixtures and demo scripts portable at all.
- **Record the honest claim, then settle it.** A sweep records `DELETE_REQUESTED` because that is all it knows; the receipt settles it to what actually happened. The ledger outlives the message.
- **Insert-only is enforced by absence.** There is no update or delete endpoint in the audit service. That is the whole mechanism, and it is stronger than a permission check.
- **The UI tells the truth about asynchrony.** 'Resolving', not a reassuring count of zero. 'Queued', not 'added'. A hold whose custodian scope never reaches the wire is named as such rather than rendered as an empty list.

The thread running through all of it: this is a system whose worst failure is not downtime but *quiet, confident, incorrect action*. Almost every design decision above trades availability or convenience for the ability to say, afterwards and with evidence, exactly what happened.